

Week 5 Reflection
TCSS-498 Directed Reading (Raghavi Sakpal)
Seth Campbell

For this week, I've been largely working on bringing basic teleoperation online, as the underlying signal paths are foundational to how the kinematics will work. By the end of the week, I was able to bring the process online, though I had some issues to work through along the way. All of these could have been avoided by going with a prebuilt library and an Arduino solution, but I felt that would have taken away from learning the underlying principles of the project. The two main issues I struggled with were power delivery and position offsets.

To address power delivery, it's important to note that I used the official documentation from Seeed, the manufacturer of the servo adapter boards. The documentation was heavily aimed at this exact application, outlining the use of the same servos in parallel operation with a microcontroller, their example used an Arduino board. In terms of power needs, they specified a 12V supply for the servos and a separate supply for the associated microcontroller. After hours of adjusting the code and trying to figure out why my transmitted signal was being circulated back on the receiving end, I examined the electrical diagram for the board and found that inputs for both 3.3V and 5V were needed to activate the circuits required for converting half-duplex to full-duplex and handling signal transmission timing. Fortunately, the board was designed for integration with a pinned ESP32, so the 5V, VCC, and 3.3V pins were aligned on both boards. With a direct connection and re-pinning in the ServoCommunication file, the signal path was finally complete.

At this point, I was getting periodic micro movements from the follower arm; sometimes in the direction of the leader, but mostly seeming random. The signal path appeared correct, so I assumed the desired positions were being read and applied. However, when both arms were in the same physical position, the data packets were noticeably different. A quick look at the memory table revealed an EEPROM setting for position offset. After several attempts, I was able to set each arm to a resting pose and configure all servo offsets to treat those positions as a relative 180 degrees. Once that was done, I finally achieved fully articulated mirrored motion from the leader to the follower.

The next task is to bring forward kinematics online. To do this, I need to integrate the frame and transformation classes to handle the process. The Transformer class will serve as the basis for forward kinematics, while the Frame class handles the dataset. I also need to source the DH values, as the full calibration process, while accurate, will have to wait for another time.